

Student ID:

IT24101554

**BSc (Hons) in Information Technology****IT3091 - Machine Learning****Y3.S1 – 2026****Faculty of Computing**

LAB SHEET 06

Regression , SVM & SVR with Scikit-Learn

Learning Outcomes

By the end of this lab session, students will be able to:

- Implement Simple Linear Regression from scratch using NumPy.
- Calculate predictions, residuals, Mean Squared Error (MSE), and regression evaluation metrics.
- Train a Linear Regression model from scratch using Gradient Descent.
- Fit and tune a Support Vector Machine (SVM) classifier using Scikit-Learn.
- Explain the decision boundary, margin, support vectors, and Hinge Loss used in SVM, and relate them to a fitted model's own attributes.
- Fit and tune a Support Vector Regression (SVR) model using Scikit-Learn.
- Explain the ϵ -tube and ϵ -insensitive loss used in SVR, and how ϵ and C shape the fitted model.
- Compare linear and nonlinear (kernel-based) SVM/SVR behaviour, and relate the library-based models to the from-scratch regression model.

Software / Resources Required

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy, Pandas, Matplotlib and Scikit-learn

Important: Part A (Linear Regression) is still implemented from scratch using basic Python/NumPy operations, exactly as before. Parts B and C (SVM and SVR) now use Scikit-Learn directly: you will fit real models, inspect their attributes, and explore how their hyperparameters change behaviour, rather than deriving the training loop by hand.

Part A

Linear Regression from Scratch

A.1 – Dataset & Problem Setup

A university wants to predict a student's **Exam Score** based on the number of **Hours Studied**.

Student	Hours Studied (X)	Exam Score (Y)
A	1	48
B	2	52
C	3	58
D	4	64
E	5	70
F	6	75

```
import numpy as np
```

```
X = np.array([1, 2, 3, 4, 5, 6])  
y = np.array([48, 52, 58, 64, 70, 75])
```

Task 1 – Understand the Problem

1. Identify the input feature, target variable, and type of supervised learning problem.
2. Create a scatter plot of Hours Studied vs. Exam Score.
3. Describe the relationship visible in the scatter plot.
4. Why is regression more appropriate than classification for this problem?

1.)

Hours Studied , Exam Score , Supervised Learning → Regression

2.)

```
import numpy as np
import matplotlib.pyplot as plt
```

```
X = np.array([1, 2, 3, 4, 5, 6])
y = np.array([48, 52, 58, 64, 70, 75])
```

```
plt.scatter(X, y)
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Hours Studied vs Exam Score")
plt.show()
```

3.)

The scatter plot shows a strong positive approximately linear relationship between hours studied and exam score. As the number of hours studied increases, the exam score generally increases.

4.)

Regression is appropriate because the target variable, Exam Score, is a continuous numerical value. Classification is used when the target consists of discrete categories such as Pass/Fail or Disease/No Disease.

A.2 – Build the Prediction Function

Simple Linear Regression

$$\hat{y} = \beta_0 + \beta_1 x$$

β_0 = intercept β_1 = regression coefficient $\hat{y}$ = predicted value

Task 2 – Initial Predictions

1. Initialize $\beta_0 = 0$ and $\beta_1 = 0$.
2. Complete the prediction function without using Scikit-learn.
3. Generate the initial predictions for all observations.
4. Explain the roles of β_0 and β_1 .

```
def predict(X, beta0, beta1):  
    # TODO: calculate y_hat  
    pass  
  
beta0 = 0  
beta1 = 0  
  
y_hat = predict(X, beta0, beta1)
```

1.)

```
beta0 = 0
```

```
beta1 = 0
```

2.)

```
def predict(X, beta0, beta1):  
    return beta0 + beta1 * X
```

```
beta0 = 0
```

```
beta1 = 0
```

```
y_hat = predict(X, beta0, beta1)
```

```
print(y_hat)
```

3.)

β_0 — Intercept

The predicted value when: $x=0$

β_1 — Slope

Shows how much the predicted output changes when X increases by one unit.

A.3 – Residuals & Mean Squared Error

Residual: $e_i = y_i - \hat{y}_i$

Mean Squared Error: $MSE = (1/n) \sum (y_i - \hat{y}_i)^2$

Task 3 – Calculate Error from Scratch

1. Calculate the residual for every observation.
2. Implement MSE from scratch.
3. Display the actual value, predicted value and residual for each observation.
4. What do positive, negative and near-zero residuals indicate?
5. Why are errors squared when calculating MSE?

```
def mse(y, y_hat):  
    # TODO: implement Mean Squared Error  
    pass
```

1.)

Calculate residuals

```
residuals = y - y_hat
```

```
print("Residuals:")
```

```
print(residuals)
```

2.)

```
def mse(y, y_hat):
```

```
    errors = y - y_hat
```

```
    squared_errors = errors ** 2
```

```
    return np.mean(squared_errors)
```

```
mse_value = mse(y, y_hat)
```

```
print("MSE:", mse_value)
```

3.)

```
print("Actual\tPredicted\tResidual")
```

```
for actual, predicted, residual in zip(y, y_hat, residuals):
```

```
    print(f"{actual}\t{predicted}\t{residual}")
```

4.)

- **Positive residual:** The actual value is greater than the predicted value. The model **underpredicted**.

- **Negative residual:** The actual value is smaller than the predicted value. The model **overpredicted**.
- **Near-zero residual:** The predicted value is close to the actual value, meaning the prediction has **very little error**.

5.)

Errors are squared for two main reasons:

1. To prevent positive and negative errors from cancelling each other.
2. To give greater penalty to large errors.

A.4 – Train Linear Regression using Gradient Descent

$$\partial J / \partial \beta_0 = (2/n) \sum (\hat{y}_i - y_i)$$

$$\partial J / \partial \beta_1 = (2/n) \sum (\hat{y}_i - y_i) x_i$$

Update: $\beta_j := \beta_j - \alpha(\partial J / \partial \beta_j)$

Task 4 – Implement Gradient Descent

1. Set a suitable learning rate and number of iterations.
2. Calculate predictions and gradients at each iteration.
3. Update β_0 and β_1 .
4. Store the MSE at each iteration.
5. Display the final β_0 , β_1 and MSE.
6. Plot Iteration vs. MSE and explain the convergence behaviour.
7. Plot the final regression line together with the observations.

```
learning_rate = 0.01
iterations = 1000
beta0 = 0
beta1 = 0
loss_history = []

for i in range(iterations):
    y_hat = predict(X, beta0, beta1)

    # TODO: calculate dbeta0
    # TODO: calculate dbeta1
```

```
# TODO: update beta0 and beta1

# TODO: calculate and store MSE

1 , 2 , 3 , 4 , 5.)
learning_rate = 0.01
iterations = 1000

beta0 = 0.0
beta1 = 0.0

loss_history = []

n = len(X)

for i in range(iterations):

    # Prediction
    y_hat = predict(X, beta0, beta1)

    # Gradients
    dbeta0 = (2 / n) * np.sum(y_hat - y)

    dbeta1 = (2 / n) * np.sum((y_hat - y) * X)

    # Update parameters
    beta0 = beta0 - learning_rate * dbeta0
    beta1 = beta1 - learning_rate * dbeta1

    # Calculate MSE
    current_mse = mse(y, predict(X, beta0, beta1))

    loss_history.append(current_mse)

print("Final beta0:", beta0)
print("Final beta1:", beta1)
print("Final MSE:", loss_history[-1])

6.)
```

The MSE decreases as the number of iterations increases. This shows that gradient descent is updating the parameters toward values that reduce the prediction error. The curve eventually becomes relatively flat, indicating that the model is approaching convergence.

7.)

```
final_predictions = predict(X, beta0, beta1)
```

```
plt.scatter(X, y, label="Actual")
```

```
plt.plot(X, final_predictions, label="Regression Line")
```

```
plt.xlabel("Hours Studied")
```

```
plt.ylabel("Exam Score")
```

```
plt.title("Linear Regression")
```

```
plt.legend()
```

```
plt.show()
```

A.5 – Evaluate the Regression Model

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$$

Task 5 – Metrics from Scratch

1. Implement MAE, MSE, RMSE and R^2 without using Scikit-learn metrics.
2. Interpret each value.
3. Which metrics are expressed in the same units as the target?
4. Which metric penalizes larger errors more strongly?
5. Predict the exam score for a student who studies **4.5 hours**.

1.)

```
def mae(y, y_hat):
    return np.mean(np.abs(y - y_hat))
```

```
def mse(y, y_hat):
    return np.mean((y - y_hat) ** 2)
```

```
def rmse(y, y_hat):
    return np.sqrt(mse(y, y_hat))

def r2_score_manual(y, y_hat):
    ss_res = np.sum((y - y_hat) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    return 1 - (ss_res / ss_tot)

2.)
beta0_ols = 41.6666666667
beta1_ols = 5.57142857143

pred_ols = predict(X, beta0_ols, beta1_ols)

print("MAE :", mae(y, pred_ols))
print("MSE :", mse(y, pred_ols))
print("RMSE:", rmse(y, pred_ols))
print("R² :", r2_score_manual(y, pred_ols))
```

3.)
 MAE -- same units
 RMSE -- same units
 MSE is squared units
 R² has no physical unit.

4.)
 MSE
 because the errors are squared.

5.)
 $y^{\wedge}=41.6667+5.57143x$
 $y^{\wedge}=41.6667+(5.57143)(4.5)$
 ≈ 66.74

A.6 – Verify using Scikit-learn

1. Train `LinearRegression` using the same dataset.
2. Display the intercept and coefficient.
3. Predict the score for 4.5 hours.
4. Compare the results with your from-scratch model.

1 , 2 , 3)

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X.reshape(-1, 1), y)
print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
prediction = model.predict([[4.5]])
print("Prediction for 4.5 hours:", prediction[0])
```

4.)

from-scratch model and Scikit-Learn model should produce essentially the same **optimal linear regression** parameters.

Part B

Support Vector Machine (SVM) with Scikit-Learn

B.1 – Binary Classification Dataset

Use the following two-dimensional dataset to investigate how a linear SVM separates two classes.

Observation	X_1	X_2	Class
A	1	2	-1
B	2	2	-1
C	2	3	-1
D	4	4	+1
E	5	4	+1

F	5	5	+1
---	---	---	----

```
import numpy as np
```

```
X = np.array([[1, 2], [2, 2], [2, 3], [4, 4], [5, 4], [5, 5]])  
y = np.array([-1, -1, -1, 1, 1, 1])
```

Task 6 – Visualize the Classification Problem

1. Plot the two classes using a scatter plot.
2. Can the classes be separated by a straight line?
3. Sketch or plot more than one possible separating line.

1.)

```
X = np.array([  
    [1,2],  
    [2,2],  
    [2,3],  
    [4,4],  
    [5,4],  
    [5,5]  
)
```

```
y = np.array([-1,-1,-1,1,1,1])
```

```
plt.scatter(X[y == -1, 0], X[y == -1, 1], label="Class -1")  
plt.scatter(X[y == 1, 0], X[y == 1, 1], label="Class +1")
```

```
plt.xlabel("X1")  
plt.ylabel("X2")  
plt.title("SVM Classification Dataset")  
plt.legend()  
plt.show()
```

2.)

Yes.

There is a clear gap between the two groups.

For example, a line approximately around:

$$x_1 + x_2 - 6 = 0$$

can separate the groups.

But there are **many possible separating lines**.

3.)

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Scatter plot
```

```
plt.scatter(X[y == 0, 0], X[y == 0, 1], label="Class 0")
```

```
plt.scatter(X[y == 1, 0], X[y == 1, 1], label="Class 1")
```

```
# Values for x1
```

```
x1 = np.linspace(X[:, 0].min(), X[:, 0].max(), 100)
```

```
# Different separating lines
```

```
x2_1 = 5 - x1
```

```
x2_2 = 6 - x1
```

```
x2_3 = 7 - x1
```

```
plt.plot(x1, x2_1, label="x1 + x2 - 5 = 0")
```

```
plt.plot(x1, x2_2, label="x1 + x2 - 6 = 0")
```

```
plt.plot(x1, x2_3, label="x1 + x2 - 7 = 0")
```

```
plt.xlabel("x1")
```

```
plt.ylabel("x2")
```

```
plt.title("Multiple Possible Separating Lines")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

B.2 – SVM Decision Function

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

For two features: $f(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b$

Predict +1 when $f(\mathbf{x}) \geq 0$; otherwise predict -1.

Task 7 – Fit an SVC and Inspect Its Decision Function

1. Fit `SVC(kernel='linear', C=1.0)` on the dataset.
2. Retrieve `svc.coef_` and `svc.intercept_` and use them to compute $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ by hand for every observation.
3. Compare your hand-computed scores with `svc.decision_function(X)`, they should match.
4. Compare `svc.predict(X)` with the actual classes.

```
from sklearn.svm import SVC
```

```
svc = SVC(kernel='linear', C=1.0)
svc.fit(X, y)
```

```
w, b = svc.coef_[0], svc.intercept_[0]
scores = X @ w + b          # should match svc.decision_function(X)
preds = svc.predict(X)
```

```
1, 2.)
```

```
from sklearn.svm import SVC
```

```
svc = SVC(
    kernel="linear",
    C=1.0
)
```

```
svc.fit(X, y)
```

```
w = svc.coef_[0]
```

```
b = svc.intercept_[0]
```

```
print("w =", w)
```

```

print("b =", b)

3.)
scores = X @ w + b

4.)
scores = X @ w + b

svm_scores = svc.decision_function(X)

print("Hand-computed scores:")
print(scores)

print("\nSVC decision function:")
print(svm_scores)

```

B.3 – Margin & Support Vectors

Decision boundary: $w^T x + b = 0$

Margin boundaries: $w^T x + b = +1$ and $w^T x + b = -1$

Margin condition: $y_i(w^T x_i + b) \geq 1$

Task 8 – Investigate the Margin

1. Using the w and b from Task 7, plot the decision boundary ($w^T x + b = 0$) and both margin boundaries ($w^T x + b = \pm 1$).
2. Print `svc.support_vectors_` and mark these points on your plot.
3. Explain why exactly these observations are called **support vectors**.
4. Why does SVM prefer a large margin instead of simply finding any separating line?

```

1.)
import numpy as np
import matplotlib.pyplot as plt

# Get weights and bias
w = svc.coef_[0]
b = svc.intercept_[0]

```

```
# Create x1 values
x1 = np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 200)

# Calculate the three boundaries
x2_decision = -(w[0] * x1 + b) / w[1]
x2_margin_positive = -(w[0] * x1 + b - 1) / w[1]
x2_margin_negative = -(w[0] * x1 + b + 1) / w[1]

# Plot classes
plt.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    label="Class 0"
)

plt.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    label="Class 1"
)

# Plot decision boundary
plt.plot(
    x1,
    x2_decision,
    label="Decision boundary"
)

# Plot margin boundaries
plt.plot(
    x1,
    x2_margin_positive,
    linestyle="--",
    label="+1 margin"
)

plt.plot(
    x1,
    x2_margin_negative,
    linestyle="--",
```

```

    label="-1 margin"
)

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("SVM Decision Boundary and Margins")
plt.legend()
plt.grid(True)
plt.show()

```

2.)

```

print("Support vectors:")
print(svc.support_vectors_)

```

3.)

Support vectors are the critical training observations that determine the position of the optimal separating hyperplane and the margin. Changing these observations can change the decision boundary, whereas observations far from the margin generally have little or no direct influence on the boundary.

4.)

SVM prefers a large margin because it provides greater separation between the classes and can improve generalization to unseen data. Instead of choosing any line that separates the training data, SVM searches for a boundary that maximizes the distance between the classes, subject to its margin constraints.

B.4 – Hinge Loss

$$L_i = \max(0, 1 - y_i f(x_i))$$

Task 9 – Hinge Loss from the Fitted Model

1. Use `svc.decision_function(X)` as $f(x)$ for every observation.
2. Calculate the Hinge Loss $L_i = \max(0, 1 - y_i f(x_i))$ for each observation, one line of NumPy, no loop needed.
3. Compare points outside the margin, inside the margin and incorrectly classified points.

4. Which observations produce zero Hinge Loss? How does that compare with `svc.support_vectors_` from Task 8?

```
scores = svc.decision_function(X)
hinge = np.maximum(0, 1 - y * scores)
```

1.)

```
scores = svc.decision_function(X)
```

2.)

```
hinge = np.maximum(0, 1 - y * scores)
```

3.)

```
print("Support vector indices:")
print(svc.support_)
print("\nZero hinge-loss indices:")
print(np.where(hinge <= 1e-10)[0])
```

4.)

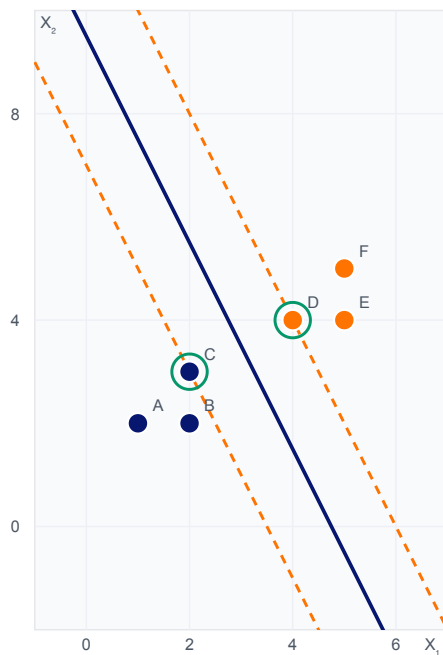
It is still important to the SVM because it helps determine the location of the optimal margin/boundary.

B.5 – Interactive Sandbox: The Effect of C on the Margin

What C Controls

C is the regularization parameter of `SVC`. A **small C** lets the model accept a wider margin even if that means a few points fall inside it; a **large C** forces the model to fit the training points as tightly as possible, shrinking the margin. Drag the slider below to watch the decision boundary, both margin boundaries and the set of support vectors change, computed from a real `SVC(kernel='linear', C=...)` fit on the exact dataset from B.1.

Task 10 – The Effect of C on the Margin



Shown at $C = 1$ (the converged hard-margin fit). Use the slider on screen to see other values of C .

C

1

Margin width ($2/\|w\|$)

2.24

Support vectors

C, D

Class -1 Class +1 (ringed points are support vectors)

1. Slide C from its smallest to its largest value. What happens to the margin width, and why?
2. How does the number of support vectors change as C increases? Connect this to what a support vector is.
3. Between $C = 0.3$ and $C = 1$ the margin barely changes even though C increases. Why does that happen for this dataset?
4. If you were worried this dataset had a mislabelled point, would you choose a smaller or a larger C ? Explain.

1.)

As C increases, the margin generally becomes narrower because the model places a larger penalty on training points that violate the margin.

2.)

As C increases from 0.05 to 0.3, the number of support vectors decreases from 6 to 2. At $C = 1$ it remains 2 for this dataset.

3.)

The dataset is already linearly separable with the same critical points C and D determining the maximum-margin solution. Increasing C further does not change the active constraints, so the fitted boundary and margin remain the same.

4.)

Choose smaller C

A smaller C allows the model to tolerate some training errors instead of bending the boundary strongly to fit every observation.

Part C

Support Vector Regression (SVR) with Scikit-Learn

C.1 – Reuse the Regression Dataset

Use the **Hours Studied** → **Exam Score** dataset from Part A. Unlike SVM classification, SVR predicts a continuous value.

Linear SVR Function

$$f(x) = wx + b$$

Instead of trying to minimize every small error, SVR defines an acceptable error region around the regression function.

Task 11 – Fit an SVR and Inspect Its Predictions

1. Fit `SVR(kernel='linear', C=1.0, epsilon=1.0)` on the dataset.
2. Retrieve `svr.coef_` and `svr.intercept_` and use them to compute $f(x) = wx + b$ by hand for every observation.
3. Compare your hand-computed values with `svr.predict(X)`, they should match.
4. Plot the observations together with the fitted line.

```
from sklearn.svm import SVR

svr = SVR(kernel='linear', C=1.0, epsilon=1.0)
svr.fit(X, y)

w, b = svr.coef_[0][0], svr.intercept_[0]
preds = w * X.ravel() + b          # should match svr.predict(X)
```

```
1, 2)
from sklearn.svm import SVR

X_svr = X.reshape(-1, 1)
```

```
svr = SVR(  
    kernel="linear",  
    C=1.0,  
    epsilon=1.0  
)  
  
svr.fit(X_svr, y)  
  
w = svr.coef_[0][0]  
b = svr.intercept_[0]  
  
preds = svr.predict(X_svr)  
  
print("w:", w)  
print("b:", b)  
print("Predictions:", preds)  
  
3.)  
preds_sklern = svr.predict(X)  
print(preds_sklern)  
  
4.)  
import matplotlib.pyplot as plt  
  
plt.scatter(X, y, label="Observed data")  
  
plt.plot(  
    X.ravel(),  
    preds_hand,  
    label="SVR fitted line"  
)  
  
plt.xlabel("Hours Studied")  
plt.ylabel("Exam Score")  
plt.title("Linear Support Vector Regression")  
plt.legend()  
plt.show()
```

C.2 – Construct the ϵ -Tube

Upper boundary: $f(x) + \epsilon$ **Lower boundary:** $f(x) - \epsilon$

An observation whose residual falls inside the tube ($|y - f(x)| < \epsilon$) contributes zero loss.

scikit-learn calls the observations that end up *on* or *outside* the tube boundary the **support vectors**.

Task 12 – Visualize the ϵ -Tube

1. Using the w and b from Task 11, and $\epsilon = 1.0$, compute the upper boundary $f(x) + \epsilon$ and lower boundary $f(x) - \epsilon$.
2. Plot the regression line, both ϵ boundaries and the actual observations.
3. Print `svr.support_` and mark these observations on your plot.
4. Which observations fall strictly inside the tube, and which sit on or outside its boundary?

```
upper = svr.predict(X) + svr.epsilon
lower = svr.predict(X) - svr.epsilon
inside = np.abs(y - svr.predict(X)) < svr.epsilon
```

```
1.)
import numpy as np
import matplotlib.pyplot as plt

# Epsilon value
epsilon = 1.0

# Predictions from the trained SVR model
y_pred = svr.predict(X)

# Calculate epsilon-tube boundaries
upper = y_pred + epsilon
lower = y_pred - epsilon

# Identify observations strictly inside the tube
inside = np.abs(y - y_pred) < epsilon

# Identify observations on or outside the tube
outside_or_boundary = ~inside

print("Predicted values:")
```

```
print(y_pred)

print("\nUpper boundary:")
print(upper)

print("\nLower boundary:")
print(lower)

print("\nInside the epsilon-tube:")
print(inside)

print("\nOn or outside the epsilon-tube:")
print(outside_or_boundary)

2.)
# Sort X values for a smooth regression line
sort_idx = np.argsort(X.ravel())

X_sorted = X.ravel()[sort_idx]
y_pred_sorted = y_pred[sort_idx]
upper_sorted = upper[sort_idx]
lower_sorted = lower[sort_idx]

# Plot actual observations
plt.figure(figsize=(10, 6))

plt.scatter(
    X,
    y,
    label="Actual observations"
)

# Regression line
plt.plot(
    X_sorted,
    y_pred_sorted,
    label="SVR regression line"
)

# Upper epsilon boundary
```

```
plt.plot(
    X_sorted,
    upper_sorted,
    linestyle="--",
    label="Upper boundary (+ $\epsilon$ )"
)

# Lower epsilon boundary
plt.plot(
    X_sorted,
    lower_sorted,
    linestyle="--",
    label="Lower boundary (- $\epsilon$ )"
)

plt.xlabel("X")
plt.ylabel("y")
plt.title("SVR Regression Line and  $\epsilon$ -Tube")
plt.legend()
plt.grid(True)

plt.show()

3.)
# Get support vector indices
support_indices = svr.support_

print("Support vector indices:")
print(support_indices)

# Get support vector X and y values
X_support = X[support_indices]
y_support = y[support_indices]

print("\nSupport vector X values:")
print(X_support)

print("\nSupport vector y values:")
print(y_support)
```

```

4.)
inside_indices = np.where(inside)[0]
boundary_indices = np.where(~inside)[0]

print("Observations strictly inside the epsilon-tube:")
print(inside_indices)

print("\nObservations on or outside the epsilon-tube:")
print(boundary_indices)

print("=== Observations strictly inside the tube ===")

for i in inside_indices:
    print(
        f"Index {i}: "
        f"X = {X[i]}, "
        f"Actual y = {y[i]:.4f}, "
        f"Predicted y = {y_pred[i]:.4f}, "
        f"Residual = {y[i] - y_pred[i]:.4f}"
    )

print("\n=== Observations on or outside the tube ===")

for i in boundary_indices:
    print(
        f"Index {i}: "
        f"X = {X[i]}, "
        f"Actual y = {y[i]:.4f}, "
        f"Predicted y = {y_pred[i]:.4f}, "
        f"Residual = {y[i] - y_pred[i]:.4f}"
    )

```

C.3 – ϵ -Insensitive Loss

$$L_{\epsilon}(y, f(x)) = \max(0, |y - f(x)| - \epsilon)$$

Task 13 – ϵ -Insensitive Loss from the Fitted Model

1. Use `svr.predict(X)` as $f(x)$ for every observation.
2. Calculate the ϵ -insensitive loss $L_\epsilon = \max(0, |y - f(x)| - \epsilon)$ for each observation, one line of NumPy, no loop needed.
3. Display Actual, Predicted, Absolute Error and Loss for each observation.
4. Which observations produce zero loss? How does that compare with `svr.support_` from Task 12?

```
preds = svr.predict(X)
loss = np.maximum(0, np.abs(y - preds) - svr.epsilon)
```

1.)

```
preds = svr.predict(X)
```

```
print("Predicted values:")
```

```
print(preds)
```

2.)

```
preds = svr.predict(X)
```

```
loss = np.maximum(
    0,
    np.abs(y - preds) - svr.epsilon
)
```

```
print("ε-insensitive loss:")
```

```
print(loss)
```

3.)

```
import pandas as pd
```

```
import numpy as np
```

```
# Predictions
```

```
preds = svr.predict(X)
```

```
# Absolute error
```

```
absolute_error = np.abs(y - preds)
```

```
# Epsilon-insensitive loss
```

```
loss = np.maximum(
    0,
```

```
        absolute_error - svr.epsilon
    )

# Create table
results = pd.DataFrame({
    "Actual": y,
    "Predicted": preds,
    "Absolute Error": absolute_error,
    "Loss": loss
})

print(results)

4.)

# Calculate predictions
preds = svr.predict(X)

# Calculate epsilon-insensitive loss
loss = np.maximum(
    0,
    np.abs(y - preds) - svr.epsilon
)

# Find observations with zero loss
zero_loss_indices = np.where(loss == 0)[0]

# Get support vector indices
support_indices = svr.support_

# Print results
print("Zero-loss observation indices:")
print(zero_loss_indices)

print("\nSupport vector indices:")
print(support_indices)

# Compare the two sets
common_indices = np.intersect1d(
    zero_loss_indices,
```

```

    support_indices
)

print("\nZero-loss observations that are also support vectors:")
print(common_indices)

# Display detailed comparison
print("\n--- Detailed Comparison ---")

for i in range(len(y)):
    is_zero_loss = loss[i] == 0
    is_support = i in support_indices

    print(
        f"Index {i}: "
        f"Actual = {y[i]:.4f}, "
        f"Predicted = {preds[i]:.4f}, "
        f"Absolute Error = {abs(y[i] - preds[i]):.4f}, "
        f"Loss = {loss[i]:.4f}, "
        f"Zero Loss = {is_zero_loss}, "
        f"Support Vector = {is_support}"
    )

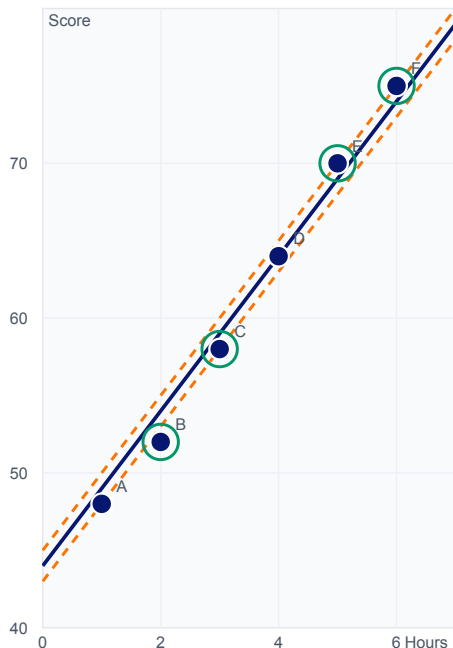
```

C.4 – Interactive Sandbox: The Effect of ϵ on the Tube

What ϵ Controls

ϵ is the width of the no-penalty tube around the fitted line in SVR. A **small** ϵ forces the line to pass close to almost every point, so nearly all of them end up as support vectors; a **large** ϵ lets many points sit comfortably inside a wide tube with zero loss, leaving fewer points to constrain the line, so it is free to become flatter. Drag the slider below to watch the fitted line, both tube boundaries and the set of support vectors change, computed from a real `SVR(kernel='linear', C=1.0, epsilon=...)` fit on the exact dataset from C.1.

Task 14 – The Effect of ϵ on the Tube

 ϵ

1

Slope (w)

5.00

Support vectors

B, C, E, F

Observation (ringed points are support vectors, on or outside the tube)

Shown at $\epsilon = 1$ (a representative middle value). Use the slider on screen to see other values of ϵ .

1. Slide ϵ from its smallest to its largest value. What happens to the width of the tube, and why?
2. How does the number of support vectors change as ϵ increases? Connect this to which points fall inside versus outside the tube.
3. The fitted line keeps getting flatter between $\epsilon = 1$ and $\epsilon = 3$, even though the number of support vectors stays the same. Why does the line keep changing?
4. If this dataset contained a genuine outlier, would a larger or a smaller ϵ make the fitted line less sensitive to it? Explain.

1.)

The ϵ -tube becomes wider because the acceptable error region around the regression line increases.

2.)

The number of support vectors generally decreases because more observations can fall inside the wider ϵ -tube and therefore do not constrain the regression function.

3.)

The support-vector count alone does not completely determine the fitted line. Their positions and the optimization constraints also matter. When ϵ changes, the allowable loss region changes, so the optimization problem changes even if the same observations remain support vectors.

4.)

Larger ϵ

A larger ϵ creates a wider no-penalty region, making the model less sensitive to moderate deviations from the fitted line.

Part D

Model Comparison & Understanding

Task 15 – Compare the Models

Property	Linear Regression	SVM	SVR
Target type	Continuous	Categorical	Continuous
Main purpose	Predict numerical values	Classification	Predict numerical values
Output	Numerical value	Class label	Numerical value
Main loss/error concept	Squared error / MSE	Hinge loss	ϵ -insensitive loss
Margin / ϵ -tube	No	Margin	ϵ -tube
Role of support vectors	None	Points defining/influencing decision boundary	Points on/outside ϵ -tube that constrain regression

1. Why can Linear Regression not be directly used for the classification problem in Part B?
2. What is the main conceptual difference between SVM and SVR?
3. Why is feature scaling important when using SVM and SVR?
4. Give one suitable real-world application for Linear Regression, SVM and SVR.

1.)

Linear Regression produces continuous numerical outputs, whereas the SVM problem requires discrete class labels. Although a threshold could be applied to regression output, ordinary Linear Regression does not directly optimize a classification objective such as maximizing the class-separating margin and minimizing classification violations.

2.)

SVM is primarily used for classification and searches for a decision boundary with a large margin between classes. SVR extends the support-vector concept to regression by finding a function while allowing errors within an ϵ -tube without penalty.

3.)

Feature scaling is important because SVM and SVR depend on geometric relationships between observations and the separating/regression function. Features with much larger numerical scales can dominate the optimization and distort the margin or kernel calculations. Scaling places features on comparable ranges.

4.)

Linear Regression

Predict house prices based on area, number of rooms and location-related numerical features.

SVM

Classify emails as spam or non-spam.

SVR

Predict electricity consumption based on temperature, time and historical usage.

End of Lab Sheet 06

Ensure that your notebook contains the completed from-scratch implementations,

requested plots, observations, metric values, predictions and the Scikit-learn verification results.



FACULTY OF COMPUTING